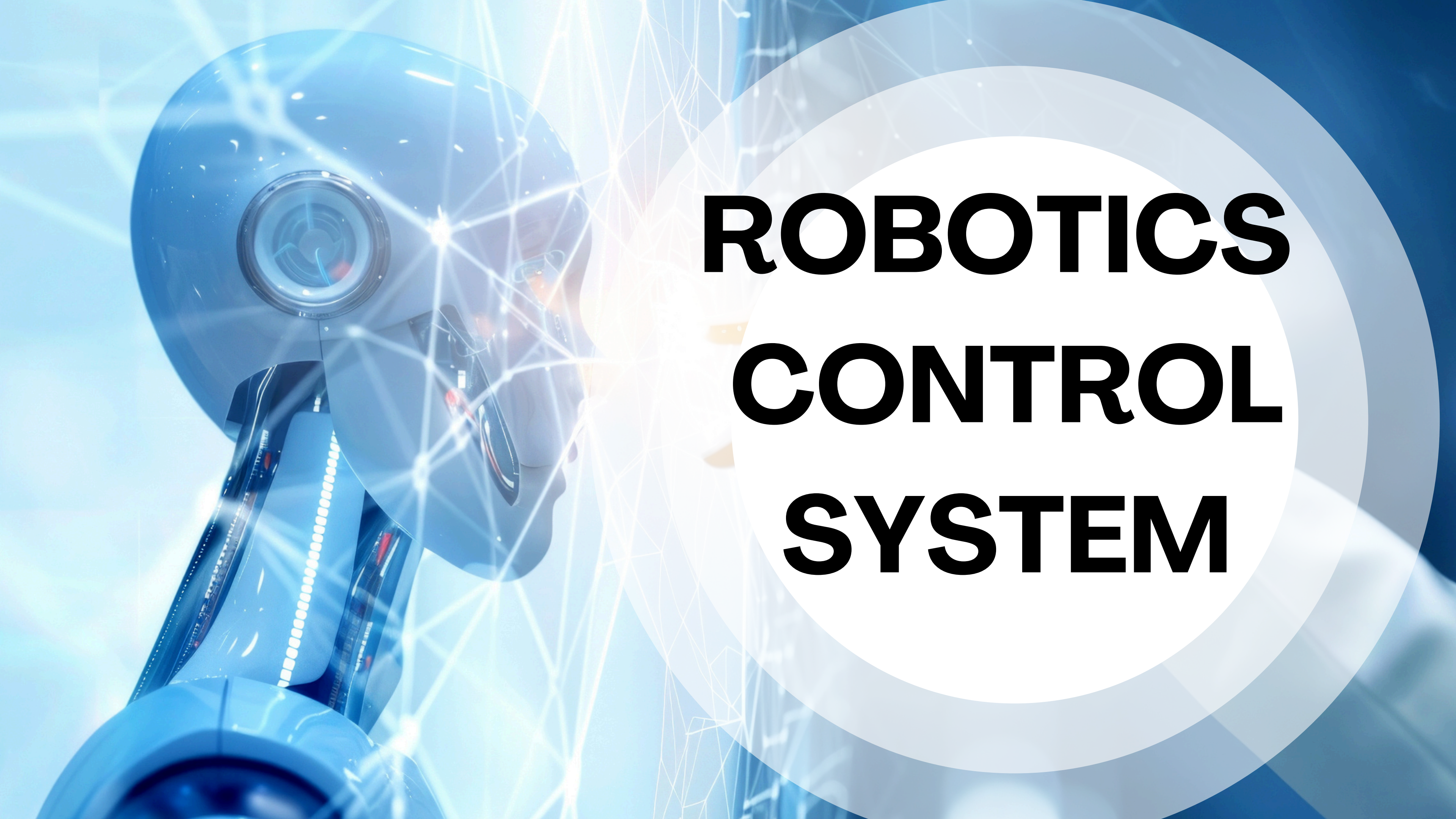




ROBOTICS CONTROL SYSTEM

PROBLEM STATEMENT:

- The system must support:
- Add new robots to the system
- Search for existing robots
- Update robot configuration and control parameters
- Delete robot records from the system
- Display robots in sorted order (e.g., by ID, status, or priority)
- Efficient management of robot data is crucial for real-time control and coordination in robotic systems



REAL-WORLD APPLICATION:

- This system can be used in:
- Industrial robot management systems
- Autonomous robot scheduling systems
- Robotics databases and monitoring systems
- Warehouse automation and logistics robots
- ✓ Helps in fast data retrieval
- ✓ Improves efficiency in real-time robotic operations



DATA STRUCTURE -USED BST:

What is BST?

A Binary Search Tree (BST) is a hierarchical data structure used to store and organize robot data efficiently.

🧩 Why BST for This Project?

⚡ Fast Search → Quickly locate robots or their status

🔄 Efficient Insert/Delete → Easy addition or removal of robots dynamically

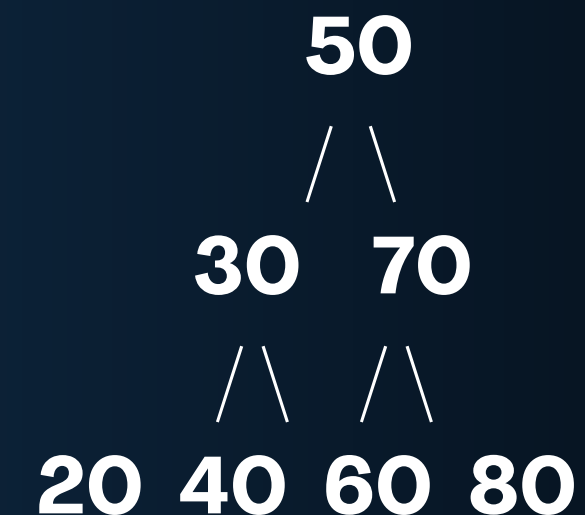
📊 Sorted Output → In-order traversal gives robots in sorted order (by ID/priority)

🧠 Memory Efficient → Uses dynamic memory allocation for flexible control

KEY ADVANTAGE:

✓ **REDUCES TIME COMPLEXITY
COMPARED TO LINEAR SEARCH**

✓ **IDEAL FOR REAL-TIME ROBOT
CONTROL AND MONITORING
SYSTEMSH TEXT**



SYSTEM DESIGN:

1. USER INTERFACE:

- MENU-BASED INPUT/OUTPUT
- ALLOWS USER TO SELECT ROBOT OPERATIONS

2. BST STORAGE:

- STORES ROBOT DATA (ROBOT ID, TASK/STATUS)
- MAINTAINS SORTED ORDER AUTOMATICALLY

3. PROCESSING MODULES:

- INSERT ROBOT
- DELETE ROBOT
- UPDATE ROBOT TASK/STATUS
- SEARCH ROBOT
- DISPLAY ROBOTS

WORKING FLOW:

USER INPUT → MENU SELECTION → FUNCTION EXECUTION → BST OPERATION → ROBOT CONTROL / DATA OUTPUTT

1.

KEY FEATURES:

- ✓ FAST SEARCHING AND UPDATING OF ROBOT DATA
- ✓ SORTED DATA USING IN-ORDER TRAVERSAL (BY ROBOT ID / PRIORITY)
- ✓ DYNAMIC MEMORY ALLOCATION FOR FLEXIBLE ROBOT MANAGEMENT
- ✓ MODULAR STRUCTURE USING FUNCTIONS FOR EASY CONTROL AND MAINTENANCE

CONCEPTS USED:

1. FUNCTIONS:

- **CREATENODE()** → CREATE NEW ROBOT NODE
- **INSERT()** → ADD A NEW ROBOT
- **SEARCH()** → FIND A ROBOT BY ID
- **UPDATE()** → UPDATE ROBOT TASK/STATUS
- **FINDMIN()** → FIND ROBOT WITH SMALLEST ID/PRIORITY
- **DELETENODE()** → REMOVE A ROBOT
- **DISPLAY()** → SHOW ALL ROBOTS

✓ **TOTAL FUNCTIONS: 7**

2. STRUCTURE:

```
STRUCT NODE {  
    INT ROBOTID;  
    INT TASKID; // OR STATUS  
    STRUCT NODE *LEFT, *RIGHT;  
};
```

◆ 3. LOOPS

- **WHILE(1)** → MENU LOOP FOR CONTINUOUS ROBOT OPERATIONS
- **RECURSION** → TREE TRAVERSAL (SEARCH, INSERT, DISPLAY ROBOTS)
✓ **USED FOR REPEATING AND MANAGING REAL-TIME OPERATIONS**

4. POINTERS:

USED FOR:

- DYNAMIC MEMORY ALLOCATION (MALLOC)
- LINKING ROBOT NODES
- TREE TRAVERSAL (NAVIGATING BETWEEN ROBOTS)
✓ CORE OF BST IMPLEMENTATION

5. VARIABLES:

- INTEGER → ROBOTID, TASKID/STATUS, CHOICE
- POINTER → ROOT, TEMP
✓ USED FOR STORING AND CONTROLLING ROBOT DATA

6. SWITCH-CASE:

- 1 INSERT ROBOT
- 2 Delete RobOT
- 3 UPDATE ROBOT
- 4 SEARCH ROBOT
- 5 DISPLAY ROBOTS
- 6 EXIT

TIME COMPLEXITY:

7. BINARY SEARCH TREE (BST)

- ✓ Advantages:
 - Fast searching of robots by ID or priority
 - Automatic sorting of robot data
 - Efficient insertion and deletion of robots in real-time

TIME COMPLEXITY:

• OPERATION COMPLEXITY	
• INSERT	$O(\log N)$
• SEARCH	$O(\log N)$
• DELETE	$O(\log N)$
• DISPLAY	$O(\log N)$

ALGORITHM:

◆ INSERTION (ROBOT)

1. Compare the robot ID with the root node
2. Insert into LEFT SUBTREE IF SMALLER, RIGHT IF LARGER
3. REPEAT UNTIL THE CORRECT POSITION IS FOUND

◆ SEARCHING (ROBOT)

4. START FROM THE ROOT NODE
5. MOVE LEFT OR RIGHT BASED ON ROBOT ID COMPARISON
6. STOP WHEN THE ROBOT IS FOUND OR NODE BECOMES NULL

◆ UPDATE (ROBOT)

1. SEARCH FOR THE ROBOT USING ROBOT ID
2. IF FOUND, MODIFY THE TASK/STATUS
3. IF NOT FOUND, DISPLAY AN ERROR MESSAGE

◆ TRAVERSAL (DISPLAY)

4. USE IN-ORDER TRAVERSAL (LEFT → ROOT → RIGHT) TO VISIT NODES IN SORTED ORDER
5. DISPLAYS ALL ROBOT RECORDS CLEARLY (SORTED BY ID/PRIORITY)

IMPLEMENTATION:

CORE CONCEPTS USED

- **STRUCTURES** → TO REPRESENT ROBOT RECORDS
- **POINTERS** → FOR DYNAMIC MEMORY & LINKING NODES
- **FUNCTIONS** → MODULAR AND REUSABLE CODE
- **RECURSION** → EFFICIENT TREE TRAVERSAL
- **SWITCH-CASE** → MENU-DRIVEN USER INTERFACE

KEY FUNCTIONAL MODULES

- **CREATENODE()** → CREATES A NEW ROBOT RECORD
- **INSERT()** → ADDS A ROBOT INTO BST
- **SEARCH()** → FINDS A ROBOT BY ROBOT ID

KEY FUNCTIONAL MODULES (CONTINUED)

- **update()** → Modifies robot task/status
- **deleteNode()** → Removes a robot
- **findMin()** → Finds in-order successor (smallest robot ID in subtree)
- **display()** → Shows sorted robot list

System Features

- ✓ Dynamic memory allocation using malloc()
- ✓ Efficient data handling using BST
- ✓ Sorted output using in-order traversal
- ✓ User-friendly menu-driven interface

CONCLUSION:

This project demonstrates the effective use of a Binary Search Tree (BST) for managing robot data in a robotics control system. It ensures:

- ✓ Faster data retrieval compared to traditional methods
- ✓ Automatic sorting using in-order traversal
- ✓ Efficient organization of robot information

The system is:

- ✓ Simple to implement
- ✓ Scalable for moderate-sized robot networks
- ✓ Suitable for real-time robotics control and monitoring systems





THANK YOU

TEAM BY:

P.JESWITH VENKATA SAI-1819

M.LAKSHAN-1787